

ui.color_input 全面详解

`ui.color_input` 是 NiceGUI 框架中基于 Quasar 的 `QInput` 组件扩展的颜色选择输入组件，核心功能是为用户提供直观的颜色选择交互，支持自定义标签、默认值、颜色变更回调等特性，广泛适用于需要颜色配置的界面场景（如主题设置、样式编辑器等）。

一、核心初始化参数

初始化 `ui.color_input` 时可配置以下关键参数，用于定义组件的基础行为和外观：

参数名	说明
<code>label</code>	组件的显示标签，用于提示用户组件功能（如 "选择颜色"）
<code>placeholder</code>	未选择颜色时显示的占位文本（如 "点击选择颜色"）
<code>value</code>	初始颜色值，支持标准颜色格式（如十六进制 <code>#000000</code> 、RGB <code>rgb(0,0,0)</code> 等）
<code>on_change</code>	颜色值变更时触发的回调函数，参数为事件对象（ <code>e.value</code> 可获取选中颜色）
<code>preview</code>	是否将按钮背景设置为选中颜色（默认值 <code>False</code> ，设为 <code>True</code> 时实时预览）

基础使用示例

```
from nicegui import ui

# 创建标签元素，用于演示颜色变更效果
label = ui.label('点击下方选择器改变我的颜色！')

# 初始化颜色选择器
ui.color_input(
    label='文本颜色',          # 组件标签
    placeholder='选择颜色',    # 占位文本
    value='#000000',           # 初始颜色（黑色）
    preview=True,              # 启用按钮背景预览选中颜色
    on_change=lambda e: label.style(f'color:{e.value}') # 颜色变更时更新标签颜色
)

ui.run()
```

二、组件核心属性

`ui.color_input` 继承并扩展了 NiceGUI 基础元素的属性，支持动态绑定、样式自定义等能力，关键属性如下：

属性名	类型	说明
<code>classes</code>	<code>Classes[Self]</code>	组件的 HTML 类名，用于通过 Tailwind/Quasar 样式自定义外观
<code>client</code>	<code>Client</code>	组件所属的客户端实例（用于多客户端场景的隔离）

属性名	类型	说明
<code>enabled</code>	<code>BindableProperty</code>	组件是否启用（可绑定，支持动态启用 / 禁用）
<code>html_id</code>	<code>str</code>	组件在 HTML DOM 中的唯一 ID（2.16.0 版本新增）
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除（只读）
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件（只读，用于判断事件响应状态）
<code>label</code>	<code>BindableProperty</code>	组件标签（支持动态绑定，可通过 <code>set_label</code> 方法修改）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性，用于扩展底层功能（如 <code>dense</code> 紧凑模式）
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式（如 <code>width: 200px</code> ）
<code>value</code>	<code>BindableProperty</code>	当前选中的颜色值（支持动态绑定，可通过 <code>set_value</code> 方法修改）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态显示 / 隐藏）

属性操作示例

```
# 动态修改颜色选择器的标签和值
color_picker = ui.color_input(label='初始标签', value='#ff0000')

# 按钮点击时更新选择器属性
ui.button('修改标签和颜色', on_click=lambda: (
    color_picker.set_label('更新后的标签'),
    color_picker.set_value('#00ff00') # 切换为绿色
))
```

三、核心方法

`ui.color_input` 提供丰富的方法用于组件交互和状态管理，关键方法分类如下：

1. 绑定相关方法

支持将组件的 `enabled`、`label`、`value`、`visible` 等属性与目标对象的属性进行单向 / 双向绑定，实现数据同步。

方法名	说明
<code>bind_enabled(target_object, ...)</code>	双向绑定组件启用状态与目标对象属性
<code>bind_enabled_from(...)</code>	单向绑定（目标对象 → 组件）启用状态
<code>bind_enabled_to(...)</code>	单向绑定（组件 → 目标对象）启用状态
<code>bind_label(...)</code>	双向绑定组件标签与目标对象属性

方法名	说明
<code>bind_label_from(...)</code>	单向绑定（目标对象 → 组件）标签
<code>bind_label_to(...)</code>	单向绑定（组件 → 目标对象）标签
<code>bind_value(...)</code>	双向绑定组件颜色值与目标对象属性（核心绑定方法）
<code>bind_value_from(...)</code>	单向绑定（目标对象 → 组件）颜色值
<code>bind_value_to(...)</code>	单向绑定（组件 → 目标对象）颜色值
<code>bind_visibility(...)</code>	双向绑定组件可见性与目标对象属性

绑定示例

```
from dataclasses import dataclass
from nicegui import ui

# 定义数据类（用于绑定）
@dataclass
class AppState:
    text_color: str = '#000000' # 与颜色选择器绑定的属性

state = AppState()

# 颜色选择器与 state.text_color 双向绑定
ui.color_input(label='绑定示例', value='#000000').bind_value(state, 'text_color')

# 实时显示绑定的颜色值
ui.label().bind_text_from(state, 'text_color', lambda color: f'当前选中颜色: {color}')
```

2. 状态操作方法

用于直接修改组件的状态（启用 / 禁用、显示 / 隐藏、值更新等）：

方法名	说明	
<code>enable()</code>	启用组件（允许用户交互）	
<code>disable()</code>	禁用组件（禁止用户交互）	
<code>set_enabled(value: bool)</code>	动态设置组件启用状态（ <code>True</code> 启用， <code>False</code> 禁用）	
<code>set_label(label: str)</code>	<code>None</code>	修改组件标签（传 <code>None</code> 隐藏标签）
<code>set_value(value: Any)</code>	设置组件的颜色值（支持任意标准颜色格式）	

方法名	说明	
<code>set_visibility(visible: bool)</code>	动态设置组件可见性（ <code>True</code> 显示， <code>False</code> 隐藏）	
<code>open_picker()</code>	主动打开颜色选择器弹窗（无需用户点击组件）	
<code>clear()</code>	清除所有子元素（极少用于颜色选择器，适用于嵌套场景）	
<code>delete()</code>	删除组件及其所有子元素（释放资源）	
<code>update()</code>	强制在客户端更新组件状态（用于手动同步数据）	

状态操作示例

```
color_picker = ui.color_input(label='可控制选择器', value='#1e90ff')

# 按钮控制颜色选择器状态
ui.button('禁用选择器', on_click=color_picker.disable)
ui.button('启用选择器', on_click=color_picker.enable)
ui.button('显示红色', on_click=lambda: color_picker.set_value('#ff0000'))
ui.button('打开选择器', on_click=color_picker.open_picker)
```

3. 其他实用方法

方法名	说明
<code>tooltip(text: str)</code>	为组件添加 tooltip 提示（鼠标悬浮时显示文本）
<code>add_resource(path)</code>	为组件添加资源（如 CSS/JS 文件，用于自定义样式 / 功能）
<code>mark(*markers)</code>	为组件添加标记（用于测试查询或依赖管理）
<code>ancestors(include_self)</code>	迭代组件的祖先元素（ <code>include_self=True</code> 包含自身）
<code>descendants(include_self)</code>	迭代组件的子元素（ <code>include_self=True</code> 包含自身）

四、事件处理

1. 核心事件：颜色变更

通过 `on_change` 参数或 `on_value_change` 方法绑定颜色变更事件，两种方式等价：

```
# 方式1: 初始化时通过 on_change 参数绑定
ui.color_input(
    on_change=lambda e: print(f'颜色变更为: {e.value}')
)

# 方式2: 通过 on_value_change 方法绑定
color_picker = ui.color_input()
color_picker.on_value_change(lambda e: print(f'颜色变更为: {e.value}'))
```

2. 通用事件订阅

通过 `on()` 方法订阅其他事件（如点击、鼠标悬浮等），支持客户端 JS 处理或服务端 Python 处理：

```
# 订阅点击事件（服务端处理）
ui.color_input().on('click', lambda: print('选择器被点击'))

# 订阅鼠标悬浮事件（客户端JS处理，无需服务端交互）
ui.color_input().on(
    'mouseover',
    js_handler=(e) => e.target.style.border = "2px solid #00ff00"
)
```

五、高级特性

1. 样式自定义

通过 `classes`、`style` 或 `default_classes` 等方法自定义组件外观：

```
# 方式1: 直接设置 style（内联CSS）
ui.color_input(style='width: 300px; margin: 10px 0;')

# 方式2: 通过 classes 应用Tailwind样式
ui.color_input(classes='w-full p-2 rounded-lg border-2 border-gray-300')

# 方式3: 修改默认类（全局生效，需在组件实例化前调用）
ui.color_input.default_classes(add='shadow-md', remove='q-input')
```

2. 动态资源与插槽

- **动态资源**：通过 `add_dynamic_resource` 添加动态生成的资源（如根据颜色值生成样式）：

```
def generate_css(color):
    return f'.custom-text {{ color: {color}; }}'

color_picker = ui.color_input(value='#ff0000')
color_picker.add_dynamic_resource('custom-css', lambda:
    generate_css(color_picker.value))
```

- **插槽**：通过 `add_slot` 方法扩展组件结构（基于 Vue 插槽机制），适用于复杂布局需求：

```
color_picker = ui.color_input()
# 添加右侧辅助图标插槽
with color_picker.add_slot('append'):
    ui.icon('colorize')
```

六、版本兼容性说明

特性 / 属性	支持版本
<code>html_id</code>	2.16.0+
<code>toggle</code> 参数 (<code>default_classes</code>)	2.7.0+
<code>strict</code> 参数 (绑定方法)	3.0.0+
同时指定 Python 和 JS 事件处理 (<code>on</code> 方法)	2.18.0+

总结

`ui.color_input` 是功能完善、扩展性强的颜色选择组件，核心优势在于：

- 1. 简洁的 API 设计，支持基础颜色选择需求的快速实现；
- 2. 丰富的绑定能力，可与业务数据模型无缝同步；
- 3. 灵活的样式和事件扩展，适配复杂界面场景；
- 4. 基于 Quasar 组件，兼顾跨浏览器兼容性和交互体验。

适用于主题配置、可视化工具、表单设计器等需要用户自定义颜色的场景，结合 NiceGUI 的其他组件（如 `ui.label`、`ui.card`）可快速构建完整的颜色交互流程。